MANTIS — Modular Agent Network Testbed for Instrumentation and Security in Multi-Agent Banking Systems

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—MANTIS (Modular Agent Network Testbed for Instrumentation and Security) is a configuration-driven security, compliance, and observability testbed built directly on a real, unmodified banking multi-agent system. This paper consolidates the system's two prior papers – the original testbed [1] and its first set of post-paper extensions [2] – into a single, complete account of MANTIS as it stands today, and contributes one further extension neither prior paper covers: a toggleable Anti-Money Laundering (AML) office, ported from the legacy codebase MANTIS's banking agents were originally extracted from [3]. The complete system instruments an otherwise-unmodified front/mid/back-office banking agent system at five fixed control points through a hook bus, where an attack, failure, or defense plugin declares its target entirely in YAML and is dispatched through the identical interface regardless of kind. On top of this base, MANTIS now ships six security-mechanism defenses that close every attack in its catalog, a real toggleable integration of the independently-developed Zero Trust Backplane project – migrated in this revision to that project's own generic architecture rewrite – four additional observability exporters, three additional banking workloads plus a second institution profile, a minimal schema-driven browser UI, and an extended scenario library of 19 measured baselines and 26 attack/fault variants. We report live-verification evidence for every capability in the system – checked against its actual emitted trace or persisted database state, never a clean exit code alone – and show that this discipline catches real defects an otherwise-passing offline test suite misses entirely: live runs surfaced and let us fix two integration defects in the new AML office and one in the migrated Zero Trust integration, none of which an offline suite of 896 passing tests (879 with neither optional dependency installed, zero failures in every combination) caught on its own. Every live capability reported here is reproducible from a committed configuration file against the real backend and a real model. Code, evidence, and configurations: https://github.com/smartsystemslab-uf/MANTIS.

Index Terms—multi-agent systems, adversarial security, large language models, banking, anti-money laundering, zero trust, observability, benchmarking

Identify applicable funding agency here. If none, delete this.

I. Introduction

Large language model (LLM) agents are moving from single-turn assistants into orchestrated multi-agent systems that carry out multi-step business processes, and banking is a natural early target: recent work proposes multi-agent frameworks for financial anomaly detection [5] and credit assessment [6], each decomposing a task across specialized agents the way a bank already organizes front-, mid-, and back-office work. That same decomposition is also where new security risk enters – inter-agent communication is an exploitable surface in its own right [7], [8], and a broad survey of agentic security concludes most defenses evaluated in isolation can be bypassed by an adaptive attacker [9], [10]. No widely available testbed previously let a researcher take a realistic, domain-specific multi-agent system and run controlled adversarial experiments against it without either reimplementing the orchestration framework or editing the business logic under test [11].

This paper is a consolidation, not a third independent contribution. The original MANTIS paper [1] built the hook bus, the banking runtime adapter, five attack and failure configurations, a seven-dimension evaluator, and a benchmarking suite – and named five extensions deliberately deferred to future work. A second paper [2] implemented all five: additional observability exporters, security-mechanism (defense) plugins, additional banking workloads and a second institution profile, a minimal schema-driven UI, and – after a team decision reversed an earlier exclusion – a real, toggleable integration of the independently-developed Zero Trust Backplane project [4]. Both papers' own artifacts (code, tests, and recorded evidence) remain frozen and unmodified; this paper neither edits nor supersedes them, it reports the complete system built on top of them, plus two further developments

neither of them covers: a new AML office extension, and tracking the Zero Trust Backplane project's own subsequent architecture rewrite.

This paper's contributions comprise three parts:

- We consolidate two prior papers – the original five-control-point testbed and its five post-paper extensions – into a single, complete, reproducible account of MANTIS as it exists today, including migrating the Zero Trust Backplane integration to that independent project's own v1.0.1 generic-architecture rewrite after confirming the originally integrated release had been deleted upstream.
- We port a fourth banking domain, an AML (Anti-Money Laundering) office, from the legacy codebase MANTIS's banking agents were extracted from, preserving its detection logic unchanged and gating it behind a toggle that leaves every pre-existing configuration's root agent byte-identical when absent.
- We report live-verification evidence – checked against the system's actual emitted trace or persisted database state, not a clean exit code or a passing unit test in isolation – for every capability in the system, and show this discipline catches real defects an offline test suite alone misses: two in the new AML office and one in the migrated Zero Trust integration, each found only once the corresponding capability was actually run live.

The remainder of this paper is organized as follows. Section II summarizes the motivation and related work established in [1]. Section III describes the complete current system design – architecture, the attack/defense plugin libraries, the Zero Trust and AML extensions, and the additional workloads – without embedding experimental results in it. Section IV describes the experimental setup shared by every live result in this paper. Section V reports those results: attack and defense effectiveness, Zero Trust and AML live verification (including the real defects live verification found and fixed), additional-workload and UI evidence, the offline test suite, and a summary of findings. Section VI concludes, and a dedicated Limitations section states this paper's scope honestly rather than folding it into the results.

II. Background and Motivation

Multi-agent decomposition is a natural fit for banking because the workloads are already organized around specialized roles – monitoring, investigation, compliance sign-off, decision – and prior work demonstrates this pattern being actively explored in finance [5], [6]. Three properties of multi-agent LLM systems create risk a single-agent deployment does not have: agents ingest content produced by other agents, not only end users, so a compromised or spoofed agent's output can influence every downstream consumer [7]; routing and hand-off decisions are themselves attack surfaces, since an adversary who can intercept or relabel an inter-agent message can redirect a request to the wrong specialist or skip a required review step [8]; and tool invocation – the mechanism by which an agent actually moves money or posts a ledger entry – can be mutated in flight, a materially different threat than a model simply answering incorrectly. General red-teaming benchmarks for LLM agents [11] provide reusable attack scaffolding but evaluate against synthetic or generic tasks rather than an existing, production-shaped business workflow, and are not designed around the constraint that the business logic under test must remain completely unmodified – a constraint that matters because a testbed requiring edits to the fraud-review agent's own logic to add an experiment cannot be trusted to measure that agent's real behavior. Zero Trust architecture – never trusting a request implicitly, mediating every action through explicit policy [12], increasingly proposed specifically for agentic AI systems [13] – is the complementary governance layer Section III-D reports as a real, toggleable integration rather than the deliberately-deferred direction the first paper treated it as.

III. System Design

This section describes what MANTIS is and how each extension works. It deliberately contains no live-verification results – those, and the evidence behind every claim made here, are reported together in Section V.

A. Architecture and Hook Bus

Figure 1 shows the complete MANTIS pipeline as it stands today. An experiment is declared entirely in a YAML configuration; an orchestrator establishes a run identifier, seed, and lifecycle, then drives the banking workflow through a hook bus exposing five fixed control points – input, agent, interaction, tool, and output. Each dispatch carries a typed context (run id, trace id, workflow id, stage, source, target, payload, metadata) to every registered plugin whose declared stages match; a plugin returns continue, mutate, skip, deny, delay, or error, and a mutation threads correctly to the next plugin in the same dispatch while the fact that one occurred is recorded in context metadata, so a plugin dispatched later in the same chain – the observability plugin, by design, so it can observe every attack's effect – can distinguish a genuinely adversarial event from an ordinary one.

Beneath the hook bus, a banking runtime adapter presents one stable interface (inventory(), workflows(), build()) over four domains: front office (transaction monitoring, fraud detection, compliance, customer service, dispute filing), mid office (operations planning, representative assist, loan pre-approval), back office (end-of-day reconciliation, settlement, SAR/exception escalation), and – new in this paper – AML office (structuring/mule-ring/identity-document investigation and SAR drafting, Section III-E). The live inventory (mantis –inventory), introspected from the tool modules and agent registry rather than maintained as a hand-authored list, reports

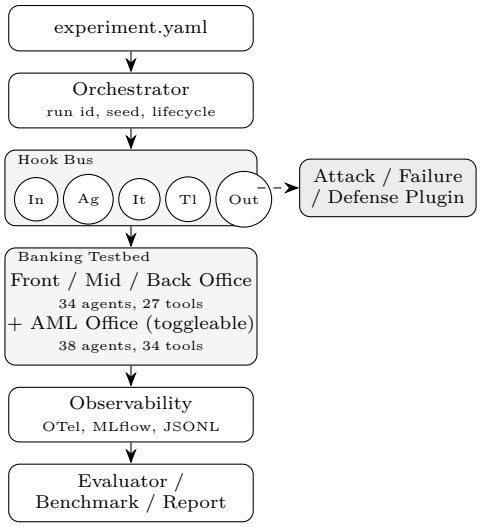


Fig. 1. The complete MANTIS pipeline. Every run passes through the same five-point hook bus regardless of banking domain; an attack, failure, or defense plugin attaches to the bus alongside the standard observability plugin. The AML office is a fourth banking domain, toggled per run, that does not change the pipeline shape when disabled.

all four domains unconditionally: 38 agents and 34 tools with the AML office registered, of which the AML office's own toggle (Section III-E) adds 4 agents and 7 tools over the 34-agent, 27-tool front/mid/back-office tree every configuration used before this paper. The AML toggle governs only whether a given run's actual agent tree instantiates those agents, not whether the inventory can discover or validate against them.

Every hook dispatch is mapped to one of seven typed event families (experiment, workflow, agent, interaction, tool, security, evaluation), exported as portable JSONL, OpenTelemetry spans, and an MLflow run sharing the same configuration hash and seed, with four additional OTLP-compatible exporters (Jaeger, Grafana Tempo, Phoenix, Langfuse) added as one-line configuration changes that fail silently rather than aborting a run when no collector is reachable. An automated evaluator reads a run's trace and reports trace completeness, tool-use correctness, workflow outcome, hook-point coverage, banking-domain coverage, artifact integrity, and attack ground truth – the last distinguishing a plugin that merely fired from one whose effect is corroborated against the configuration's own expected-tools/terminal-state baseline (this evaluator is what every result in Section V is scored by). Every run manifest records a SHA-256 configuration hash, the seed, the Python interpreter and dependency versions, the source commit, and whether the run used a mock or live model and which model served it, so a reported result always traces back to the exact code, environment, and model that produced it.

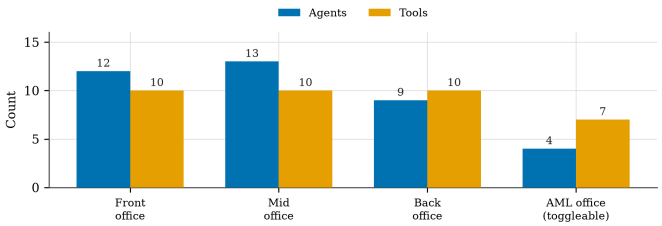


Fig. 2. Agents and tools per banking domain (from the live NativeBankingAdapter().inventory()). The AML office is the only domain whose tool count exceeds its agent count – most of its tools are read-only detection/lookup functions on a single investigator agent, rather than one tool per specialist agent as in the other three domains.

B. Attack and Failure Plugin Library

Five attack and failure configurations span all three always-on domains, registered through the identical plugin interface. Prompt injection and message spoofing both intercept the interaction control point (an agent's outgoing call to the underlying model). Route confusion intercepts a routing tool call and forces execution to a different downstream agent than the one the orchestration logic selected. Tool-parameter mutation intercepts a tool call directly and rewrites its arguments, shipped against both a front-office funds transfer and a back-office ledger post. A reliability-failure family (delay, timeout, malformed result) emits an ANOMALY security event rather than ATTACK_INJECTED, so an evaluator separates a security-caused anomaly from a routine fault by event type rather than inference. The extended scenario library (Section III-F) widens this catalog to 26 attack and fault variants under the identical interface, without changing the architecture.

C. Security-Mechanism Defenses

Six security-mechanism plugins cover every category a defense-in-depth testbed should, each registered through the identical apply(ctx) -> HookResult contract an attack plugin uses and classified in the trace as a distinct POLICY_EVENT rather than misclassified as an attack. AmountLimitGuardrailPlugin denies a tool call whose declared amount exceeds a configured limit. RiskAwareRoutingGuardPlugin independently re-fetches a transaction's real risk score from the banking backend rather than trusting the router's own decision, with an optional redirect_to recovery mode that routes to the compliant path instead of merely blocking. ResponseRedactionPlugin masks account-number-shaped tokens at the output control point. BatchIntegrityGuardPlugin independently re-checks a target batch's real expected-vs-posted totals before an end-of-day ledger update is allowed to proceed. RateLimitGuardrailPlugin caps calls per run, time window, or argument value. ActionIsolationPlugin refuses tools with a chosen side effect regardless of caller. Sec-

tion V.A reports each plugin's live-verified effect against the attack it was designed to close.

D. Zero Trust Backplane Integration

MANTIS integrates the real, independently-developed Citi_P3_Zero_Trust_Backplane project [4] – not a reimplementation – as a toggleable extension, pinned to its v1.0.1 release via a git submodule rather than an unpinned checkout, for the same reproducibility reason every other dependency here is pinned. (This revision migrates the integration to that project's own generic-architecture rewrite; Section V.B reports why, and a real defect that migration's own live verification found.) Unlike every other defense in this paper, it is not a PolicyPlugin dispatched through MANTIS's own hook bus: the real project's own generic ZeroTrustPlugin is an ADK plugin, registered directly on MANTIS's ADK App immediately after MantisHookPlugin, so every attack, policy, and observability dispatch through the hook bus still happens unconditionally before Zero Trust gets a final, independent say over whether the real tool call proceeds. A stock install never imports it, checked the same way every other optional dependency in this testbed is. A dedicated zero_trust: {enabled: true} block (or the equivalent checkbox in the minimal UI, Figure 3) turns it on or off per run; MANTIS's own contribution is a profile pointing the real project's own scanner at MANTIS's actual ADK application factory, so the signed protection bundle it builds comes from importing and introspecting MANTIS's live agent/tool graph directly, not from a hand-written policy file. A design property worth stating plainly rather than glossing over: the real scanner's placements are per-agent – which agent's own declared tools include a given call – not per-domain, which is stricter than this integration's original "agent may act on a tool in its own domain" claim: a same-domain agent that was never wired to a given tool is denied too, not only a cross-domain one. Because the AML office (Section III-E) adds a fourth sub-agent that changes the live graph's fingerprint, a second bundle is built and committed against an AML-enabled application factory; the plugin builder's own flag selects the bundle matching a run's actual graph, and loading the wrong pairing raises the real project's own staleness check rather than silently enforcing against a mismatched graph.

E. The AML Office: Porting Anti-Money Laundering from Citi_P3

MANTIS's banking agents were extracted from a larger legacy monorepo, Citi_P3 [3], during the original migration documented in MIGRATION.md. That legacy codebase's citi_banking_adk package included a working AML investigation office – an investigator agent, a SAR drafting agent, and four real detection functions – that was never carried over when the rest of the codebase was translated into MANTIS's modular, hook-bus-instrumented architecture. This section reports that port: the detection logic is adapted, not reinvented, with the mathematics and control flow of every detection function unchanged from the source; MANTIS-side work is confined to the toggle mechanism, the agent-tree wiring, and the data-access layer the ported functions call into.

Four detection functions are ported with their logic unchanged. Structuring detection (detect_structuring) evaluates a sliding window over a customer's inbound deposit-like transactions, accumulating a rolling total and reporting a hit whenever that total, within a configurable window (default 7 days), meets or exceeds a configurable threshold (default $10,000); a genuine hit creates a persisted AML alert and, optionally, a manual review case. Threshold-anomaly checking (check_threshold_anomaly) flags a transaction exceeding 5× a customer's baseline peak transfer amount. Mule-ring detection (find_mule_rings_for_customer) builds a directed graph from a customer's own transaction flows and reports closed loops via graph cycle detection, scoped – as in the source – to ring segments where the customer's own accounts appear as an endpoint. Identity-document verification (verify_identity_document) scores a submitted name (Unicode-normalized, case-folded, Levenshtein similarity), date of birth (exact calendar-date match), and ID number (exact string match) against reference values, weighted 0.30/0.30/0.40, returning a bounded, deterministic confidence score. Separately, an alert-lifecycle state machine (open → in-investigation → cleared/escalated-to-sar) is ported from the legacy codebase's standalone backend service into BankingRepository, the same SQLite-backed data-access layer every other banking tool already uses, with three new tool wrappers (get_aml_alert, list_aml_alerts, transition_aml_alert_status) giving an agent a real path to use it.

The AML office follows the same two-layer toggle discipline Section III-D established for Zero Trust, adapted to a domain rather than a policy plugin. Per-run: a dedicated aml_office: {enabled: true} configuration block (mirroring zero_trust:'s shape) controls whether the root agent's sub_agents list includes the AML office router; every configuration without that block produces a root agent whose sub_agents, instruction, and description are byte-identical to before this domain existed, verified directly by a regression test constructing the agent tree both ways and diffing it. Packaging: mule-ring detection's graph-cycle dependency (networkx) is an optional extra (pip install -e ".[aml]"), imported lazily inside the one function that needs it, so every other AML tool, and mantis –inventory/–validate, work in a stock install.

F. Additional Workloads, Minimal UI, and Scenario Library

Three additional banking workloads, each backed by real persisted records rather than a new prompt into an existing process: dispute filing (front office), SAR/exception

escalation (back office), and loan pre-approval (mid office), the last computing a real, deterministic decision from seeded account data rather than an LLM opinion. A second institution profile (regional_credit_union) reuses every agent, tool, and workflow unchanged against an isolated database and a genuinely stricter policy threshold (10% of balance versus 25%), selected by a runtime configuration field. A minimal, schema-driven browser UI (mantis –ui, Figures 3–6) provides an experiment editor and trace viewer whose every mutating action – validate, run, campaign – shells out to the identical mantis CLI subprocess a terminal user would invoke; dedicated toggle fieldsets for Zero Trust and the AML office sit alongside the generic attack/policy dropdowns. An extended scenario library ships 19 measured no-attack baselines and 26 attack/fault variants, and a from-scratch walkthrough adding a new attack plugin in about fifteen lines. Five starter campaigns – the original four [2] plus a fifth for this paper's own AML office – each a single mantis –campaign command producing a consolidated report.

IV. Experimental Setup

Every live result in Section V is produced the same way. Model. All live runs use gpt-oss-20b served via UF Navigator (https://api.ai.it.ufl.edu); the run manifest records the serving model and endpoint host for every run, never the credential, so a reported result always traces back to the exact model that produced it. Results on a different model would need separate re-runs; none are reported here. Scoring. Every reported result is read from the automated evaluator described in Section III-A (mantis –evaluate), applied to the real emitted trace of a real run – never inferred from a plugin having fired, an exit code, or a passing unit test in isolation. tool_use_correctness compares the tools a run actually called against a configuration's own expected_tools/forbidden_tools baseline; workflow_outcome compares the real terminal state reached against the configuration's expected_terminal_state; attack ground truth corroborates that an attack's effect is observable in the real trace or persisted database state, not merely that the plugin dispatched. Repetition. Attack, defense, and workload live-verification claims are repeated-trial results (5 trials per configuration unless stated otherwise); AML and Zero Trust verification claims are each a single, officially scored run per scenario from a committed configuration file, reproducible by re-running that same file. Environment. Every run manifest records a SHA-256 configuration hash, the seed, the Python interpreter and dependency versions, and the source commit (Section III-A); the offline test suite (Section V.E) runs against two independent dependency configurations (zero_trust and aml installed or not) to confirm every optional extension is genuinely opt-in.

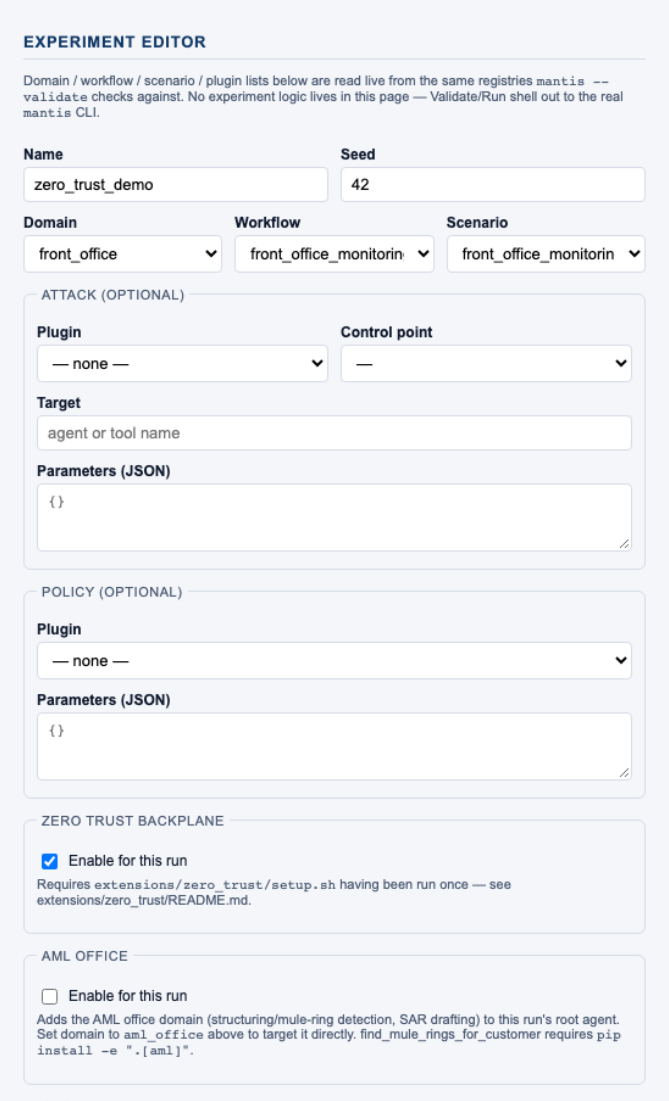


Fig. 3. The minimal UI's Experiment Editor, with the Zero Trust Backplane toggle enabled and the AML office fieldset visible below it – a real screenshot of the current UI, taken after the Zero Trust migration (Section III-D); domain/workflow/scenario lists are populated live from the same registries mantis –validate checks against, not hardcoded.

V. Experiments

A. Attack and Defense Effectiveness

Table I reports live, repeated-trial outcomes for the five base attack and failure configurations (Section III-B) against the undefended system. Route confusion is unambiguous and total: fired and produced its ground-truth effect in every trial, with the router's real transfer diverted to the chatbot workflow and compliance never reached. Tool mutation (front office) genuinely reached and altered the real call every trial, with the banking backend's own destination-account validation independently catching it every time – a defense-in-depth result, not a plugin failure. Message spoofing, prompt injection, and the malformed-

TABLE I
Live, repeated-trial outcomes for the base attack/failure library (5 trials each, undefended).

Attack/Failure	Fired	Ground-truth effect
Route confusion	5/5	5/5: real transfer diverted to chatbot workflow, compliance step skipped.
Tool mutation (front)	5/5	5/5: real call altered; backend's own validation independently caught every attempt.
Tool mutation (back)	5/5	5/5: real ledger-post call altered (undefended; see BatchIntegrityGuard, Table II).
Message spoofing	5/5	0/5 ground-truth effect: model received the spoofed message, declined to act on it.
Prompt injection	5/5	0/5 ground-truth effect: model received the injected content, declined to act on it.
Reliability failure (malformed)	5/5	0/5 ground-truth effect: model received the malformed result, recovered.

TABLE II
Live defense verification: each plugin against the attack it targets, 5 trials.

Defense	Targets	Result
AmountLimit Guardrail	Tool mutation (front): $125.50 inflated to $5,000	5/5 denied before reaching the backend
RiskAware Routing Guard	Route confusion	5/5 deny; 5/5 manual_review with the optional redirect mode
Response Redaction	Account-number leakage at output	Fixed a Unicode-hyphen detection gap; verified post-fix
Batch Integrity Guard	Tool mutation (back): ledger-post mutation	5/5 denied; 0/5 false positive on the legitimate flow
RateLimit Guardrail	Hijack-induced call loop (extended library)	0/10 undefended completions → 5/5 completed
Action Isolation	Attacked funds transfer	5/5 denied; backend transaction count unchanged

failure control each show a genuine, verified mutation reaching the real model, with the model consistently declining to act on it. The extended scenario library (Section III-F) widens this evidence to 123 scored live trials across 26 attack and fault variants without changing the architecture (Figure 4).

Table II reports each security-mechanism plugin (Section III-C) live-verified against the attack it targets, with a paired no-attack control confirming no false positive. Every attack in the shipped catalog that once succeeded undefended now has a live-verified defense.

B. Zero Trust Backplane: Live Verification and a Real Defect Found

Table III reports live verification of the migrated Zero Trust integration's access-control decisions. An agent calling its own declared tool is allowed; a same-domain agent never wired to that tool is denied, confirming the per-agent (not per-domain) design property stated in Section III-D; a cross-domain call is denied; an unrecognized agent is denied. Because the AML office adds a fourth sub-agent that changes the live graph's fingerprint, the combined AML+Zero-Trust configuration was verified against its own matching bundle, and the wrong bundle/graph pairing was confirmed to raise the real project's own staleness check rather than silently enforce against a mismatched graph. Both the base and the combined configuration now run live end to end with tool_use_correctness 1.0.

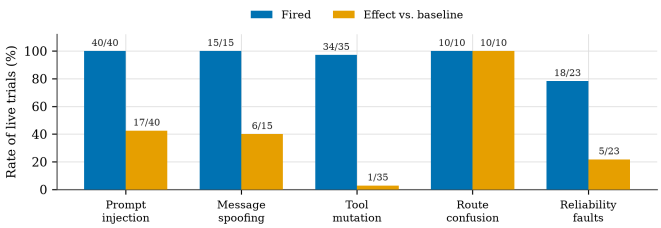


Fig. 4. The extended attack library's live trial outcomes by family (Section III-F). Nearly every family fires close to 100% of the time; whether that translates into a ground-truth-detected effect varies sharply – route confusion is unambiguous and total, while prompt injection, message spoofing, and tool mutation show the model/backend resisting most attempts.

TABLE III
Zero Trust Backplane live verification.

Test case	Outcome
Agent calls its own declared tool	Allowed
Same-domain agent, tool it was never wired to	Denied
Cross-domain tool call	Denied
Unrecognized agent	Denied
Combined AML + Zero Trust, matching bundle	Allowed/denied correctly; tool_use_correctness 1.0
Combined AML + Zero Trust, mismatched bundle	Rejected (real staleness check), as required

Finding: live verification caught a defect the unit suite also missed. The first version of this migration built the protection bundle against the application factory called with no MCP session. Every live run, however, gives the root agent a real session, which attaches one extra passthrough tool per MCP tool directly onto it – a structurally different agent graph than the one the bundle was built against. The unit test suite made the identical omission when constructing its own test fixture, and passed regardless; only running the toggle's own demo configuration live raised the real project's own staleness check immediately. This was fixed by scanning against a factory that spawns the real MCP server the same way a live run does, and the regression tests were corrected to match the same construction. This is the second, independent instance in this paper of live verification

catching what an offline-only check missed (Section V-C reports the first) – neither defect involved incorrect logic in isolation; both were properties of how the system is actually wired together at run time.

C. AML Office: Live Verification and Two Real Defects Found

Five scenarios exercise the AML office end to end against the live backend and a real model: a structuring-detection positive case (three seeded inbound wires from one counterparty summing to $12,000 within seven days, past the $10,000 threshold), a structuring negative control, a mule-ring negative control, an identity-verification case, and an alert-triage case against a seeded reference alert. The first live pass of all five surfaced two real integration defects, neither caught by the unit test suite because both are properties of the live agent tree, not of the ported functions in isolation.

Routing gap. The identity-verification scenario was misrouted to front_office_router instead of aml_office_router: the root agent's routing instruction mentioned "AML investigation, suspicious activity, and SAR drafting" but not identity verification by name, and a plausible-sounding KYC-flavored prompt was reasonably routed to the front office instead, where it called get_customer_context with the submitted ID number in place of a customer id and produced a nonsensical result. The fix was wording, not logic: the root agent's and the AML router's instructions now explicitly name identity-document verification and alert triage as AML-office tasks.

Unreachable capability. The alert-triage scenario asked to transition a seeded alert's status; transition_aml_alert_status existed, was correctly implemented, and was unit-tested directly as a function – but no agent in the built tree had it, or its sibling read tools, on its toolset at all. The three alert-lifecycle tools were wired into the existing aml_investigator agent's toolset, the single tool-using agent in this domain, and its instruction extended to describe the triage capability.

Both fixes were verified live after correction: the identity-verification scenario now routes correctly to aml_office_router → aml_investigation_workflow, calls verify_identity_document with the exact submitted/reference values from the prompt, and reports 100% match confidence. The alert-triage scenario now calls transition_aml_alert_status, and we confirmed the effect directly against the database rather than trusting the model's own report: reading the alert back afterward shows its status genuinely changed from open to in-investigation. All five scenarios were then run officially from committed configuration files (configs/extensions/aml_office_*_demo.yaml) and scored by mantis –evaluate: tool_use_correctness 1.0 and workflow_outcome 1.0 (completed) for every one of the five, with zero missing expected tools and zero forbidden tools used. Table IV summarizes all five after both fixes.



Fig. 5. The minimal UI's Trace Viewer rendering the real aml_office_demo trace – every event is real, from this run, not illustrative: the root agent's two transfer_to_agent hops into the AML office, then aml_investigator's own detect_structuring call and result.

TABLE IV
AML office live verification, all five scenarios, against a real model after the fixes above.

Scenario	Tool called	Observed
Structuring (positive)	detect_structuring	Correctly detected $12,000 cumulative over 7 days; real alert and manual-review case persisted.
Structuring (routine control)	detect_structuring	Correctly reported no structuring; no alert created.
Mule ring (control)	find_mule_rings_for_customer	Correctly reported no closed loop in the seeded history.
Identity verification	verify_identity_document	Correctly routed after fix; 100% match confidence on identical submitted/reference values.
Alert triage	transition_aml_alert_status	Correctly routed after fix; database re-read confirms the real status transition.

D. Additional Workloads, Institution Variant, and UI

Table V consolidates live evidence for the three additional banking workloads, the second institution profile, and the minimal UI (Section III-F), alongside the at-

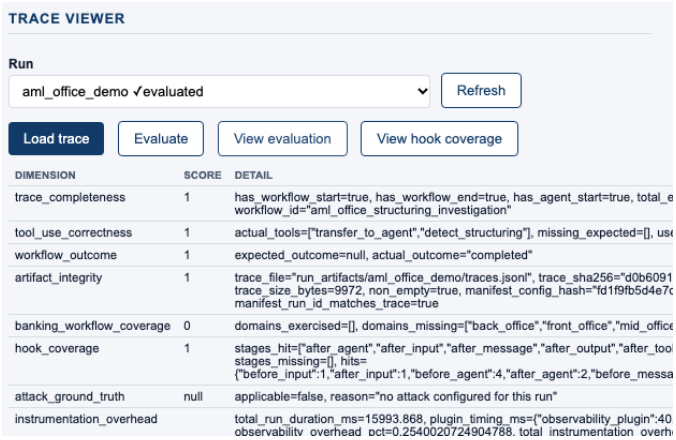


Fig. 6. The same UI, scoring the same run: tool_use_correctness and workflow_outcome both 1.0, matching the mantis –evaluate CLI output reported above exactly, since both call the identical evaluator – the UI has no separate scoring logic of its own.

TABLE V
Complete-system live-verification evidence.

Capability	Live evidence
Attacks/failures (5)	See Table I.
Defenses (6)	See Table II; no false positive on any paired no-attack control.
Zero Trust Backplane	See Table III.
AML office (new)	See Table IV; all 5 scenarios correct after fixing 2 real integration defects found via live verification.
Additional workloads (3)	Live, 5/5 each: real persisted dispute, SAR, and loan-decision records.
Institution variant	Live, 5/5: identical code path, isolated data, genuinely different real decision.
Minimal UI	Real invalid config rejected with the real CLI's own error text; real server confirmed via HTTP; Figures 3–6 are real screenshots, not mockups.
Scenario library	123 scored live trials across 26 attack/fault variants; 5 more for the AML office.

tack/defense, Zero Trust, and AML results above, as a single reference for the complete system.

E. Offline Test Suite

The offline test suite – unit and contract tests, a parametrized consistency suite over every shipped configuration and scenario, and regression guards for every toggleable extension – stands at 879 passing tests with neither optional dependency (zero_trust, aml) installed, 896 with both installed, zero failures in every combination, and 201 more skipped by design without a live backend or model. This paper's own AML office extension adds 53 of those tests (repository-layer, detection-function, toggle, and drift-regression coverage) on top of the base established by [1] and the fourteen further categories added by [2]. The Zero Trust migration's own regression guards include deliberately building the plugin against a mismatched agent graph and confirming the real project's own check rejects it (Section V-B).

F. Summary of Findings

Live verification catches what an offline-only check misses, twice, independently. The AML office's two defects and the Zero Trust migration's one defect were each invisible to a passing unit test suite because each is a property of the live agent tree or the live runtime environment, not of any single function or module in isolation – the same methodological lesson both prior papers report, now observed a third and fourth time in genuinely different subsystems.

Defense-in-depth closes every attack in the shipped catalog. Every attack that once succeeded undefended now has a live-verified defense (Table II), with zero false positives on any paired no-attack control.

A migrated real-world dependency can silently diverge from its own test suite's assumptions. The Zero Trust Backplane's own generic rewrite changed what a correctly-scanned application graph looks like; because our test fixture and the original bundle-build step made the identical simplifying assumption, nothing caught the mismatch until a real run exercised it end to end.

Per-agent enforcement is a stronger guarantee than the one originally claimed. The real Zero Trust scanner's placements are per-agent, not per-domain – every live result in Table III is consistent with the stricter claim, not only the original, looser one.

VI. Conclusion

This paper consolidates MANTIS as a complete system: the original five-control-point testbed [1], all five of its deferred extensions [2] including a real, toggleable Zero Trust Backplane integration now migrated to that project's own generic architecture, and – new here – a toggleable AML office porting real structuring, mule-ring, and identity-verification detection logic from the legacy Citi_P3 codebase MANTIS was originally extracted from. Every extension across all three papers shares the same discipline: a capability is reported as working only once verified against the system's actual emitted trace or persisted database state, not a clean exit code or a passing unit test in isolation. That discipline paid for itself directly in this revision, catching two real defects in the new AML office and a third in the migrated Zero Trust integration before any of the three could honestly be called working. The complete offline suite passes with zero failures across all four combinations of its two optional dependencies (879 with neither, 896 with both), and every live capability this paper reports is reproducible from a committed configuration file against the real backend and a real model.

Limitations

Single model evaluated. Every live result in this paper uses one model, gpt-oss-20b served via UF Navigator (Section IV); results on a different model would need separate re-runs, and we do not claim the specific numeric outcomes

(e.g. an attack's ground-truth effect rate) generalize across models, only that the testbed and its evaluator are model-agnostic. Practical, not maximal, trial counts. Repeated-trial results use 5 trials per configuration, a practical and repeatable sample size rather than one chosen to tighten confidence bounds maximally; a larger campaign along the same already-implemented axes (Section III-A's benchmarking suite) would narrow them further. The AML investigator's own SAR-drafting judgment is an inherited, not introduced, limitation. detect_structuring itself is deterministic and, as Table IV shows, correct; the investigator agent's own gating logic for whether to draft a SAR (risk score $\geq$ 50, or an investigation status containing "suspicious") is inherited unchanged from the source prompt. A live run against the positive structuring case correctly identified the pattern in its findings text but assigned a risk score of 7.2 – well under the reporter's trigger threshold – and separately populated its case_facts with invented transaction amounts and dates rather than the real values the tool had just returned. We report this as a known limitation of that prompt against this specific smaller model, inherited directly from Citi_P3 and not something this port introduced or silently corrected. Zero Trust's enforced scope is domain-scoped tool access, not routing. The integration deliberately leaves transfer_to_agent unconstrained (Section III-D) because MANTIS's own agent tree uses legitimate cross-domain routing as normal behavior; it does not extend to the real project's fuller workflow- and risk-aware policy model, which remains, as in [2], an open direction rather than one this paper closes.

Acknowledgment

References

[1] [Authors], "MANTIS – Modular Agent Network Testbed for Instrumentation and Security in Multi-Agent Banking Systems," 2026.

[2] [Authors], "MANTIS Extended – Post-Paper Extensions to a Banking Multi-Agent Security Testbed," 2026.

[3] smartsystemslab-uf, "Citi P3," github.com/smartsystemslab-uf/Citi_P3, 2026. (Legacy source repository MANTIS's banking agents were extracted from.)

[4] smartsystemslab-uf, "Citi P3 Zero Trust Backplane," release v1.0.1, github.com/smartsystemslab-uf/Citi_P3_Zero_Trust_Backplane, 2026.

[5] X. Zhang et al., "Enhancing Anomaly Detection in Financial Markets with an LLM-based Multi-Agent Framework," arXiv preprint arXiv:2403.19735, 2024.

[6] "MASCA: LLM based-Multi Agents System for Credit Assessment," arXiv preprint arXiv:2507.22758, 2025.

[7] D. Lee and M. Tiwari, "Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems," arXiv preprint arXiv:2410.07283, 2024.

[8] "Red-Teaming LLM Multi-Agent Systems via Communication Attacks," in Findings of the Association for Computational Linguistics: ACL 2025, arXiv preprint arXiv:2502.14847.

[9] "A Survey on Agentic Security: Applications, Threats and Defenses," arXiv preprint arXiv:2510.06445, 2025.

[10] "Security Considerations for Multi-agent Systems," arXiv preprint arXiv:2603.09002, 2026.

[11] "REDAgentBench: Executable Red Teaming and Faithful Measurement of LLM Agent Systems," arXiv preprint arXiv:2608.10669, 2026.

[12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, Aug. 2020.

[13] "A Novel Zero-Trust Identity Framework for Agentic AI: Decentralized Authentication and Fine-Grained Access Control," arXiv preprint arXiv:2505.19301, 2025.